# University of Colombo School of Computing
# SCS 1308 – Foundations of Algorithms
## Take – Home 02

---

## Instructions

- Try the following questions and upload your answer script as a zip file to the given link in the UGVLE on/before 25th October at 10 PM.
- It's an individual task. The deadline is 10:00 PM. Submissions will be accepted until midnight with a late penalty of 5% deducted for each hour past the deadline.
- Note: Rename your zip file as indexNo_2_last_namefname.zip.

1. A student proposes the following recurrence for a recursive algorithm:

$$T(n) = T(n/2) + n, T(1) = 1$$

The algorithm is described as dividing a problem into two equal parts, recursively processing both parts, and then performing $n$ additional operations.

a. Explain why the student's recurrence may not accurately represent the described algorithm.

b. Write the correct recurrence equation.

c. Solve the student's recurrence.

d. Solve the corrected recurrence.

e. Compare the two complexities and explain why accurately counting recursive calls is essential when deriving recurrence equations.

2. A processing system divides a problem of size $n$ into two subproblems of sizes $n/3$ and $2n/3$. It then performs $\Theta(n)$ additional work to combine the results.

a. Derive the recurrence equation for the running time.

b. Draw the first two levels of the recursion tree.

c. Explain why the recursion tree is not balanced.

d. Estimate the height of the tree along the longest recursive path.

e. Determine the asymptotic time complexity and justify your conclusion.

3. Design a recursive algorithm for a problem of your choice, other than factorial, binary search, or merge sort.

   Your algorithm must:

   - Have a clearly defined base case.
   - Make at least one recursive call.
   - Perform additional work outside its recursive calls.

   Then:

   a. Describe the problem and explain your algorithm.

   b. Write the pseudocode.

   c. Derive the recurrence equation, clearly identifying the base-case cost, recursive-call costs, and non-recursive work.

   d. Solve the recurrence using either the iteration method, recursion tree method, or substitution method.

   e. State the asymptotic time complexity and explain how the structure of the algorithm determines it.

4. Consider the following recurrence equation:

$$T(n) = 2T(n/2) + n^2, T(1) = 1$$

   Assume $n = 2^k$.

   a. Draw the recursion tree up to the third level.

   b. Calculate the total non-recursive work at each level.

   c. Determine the height of the recursion tree.

d. Calculate the total work across all levels.

e. State the asymptotic time complexity and explain which levels contribute most to the total cost.

5. Consider the following algorithm:

```
Transform(n)
  if n <= 1
    return
  for i = 1 to n
    perform one constant-time operation
  Transform(n / 2)
```

Assume integer division is used.

a. Identify the base case and recursive call.

b. Determine the cost of the loop for an input of size $n$.

c. Derive the recurrence equation for the running time.

d. Solve the recurrence using the iteration method or a recursion tree.

e. Explain why counting only the number of recursive calls would not give the complete time complexity.